

HackerRank Orchestrate - AI Judge Interview Prep

Q1: Why did you choose your specific Vision model and API over others?

We chose Groq's Vision models (like Llama 3/4 Vision) for their extremely low latency and solid zero-shot image analysis capabilities. Given the challenge's constraint to build a functional system quickly, Groq provided the best balance of speed, cost (free tier), and reasoning quality for multi-modal evidence review.

Q2: How did you handle the API rate limits and quotas?

Groq's free tier has strict Requests Per Minute (RPM) limits which initially caused '429 Too Many Requests' errors. To solve this efficiently without wasting time on blind exponential backoff, we implemented a robust GroqClient wrapper. It handles proactive round-robin key rotation across multiple API keys. We paired this with a smart 15-second per-key cooldown, yielding a steady, delay-free throughput that completely eliminated rate limit hits during our final run.

Q3: Did you encounter any image processing errors? How did you resolve them?

Yes, we encountered opaque 'Invalid Image Data' (400) and 'Payload Too Large' (413) errors. Through debugging, we discovered two issues: (1) images exceeding Groq's 30-million pixel / 20MB limits, and (2) unsupported image formats like AVIF or WEBP masquerading as .jpg files. We implemented a preprocessing layer using the Pillow library to automatically downscale oversized images via Lanczos resampling and force-convert all unsupported formats to JPEG before base64 encoding.

Q4: How did you ensure the model output was structured and parsable?

We utilized the API's native JSON mode (`response_format={"type": "json_object"}`) combined with a highly structured prompt. We defined the exact JSON schema required for the claims prediction (`claim_status`, `issue_type`, `severity`, etc.) and enforced the output structure, eliminating the need for brittle Regex parsing.

Q5: How did you utilize AI assistants during your development process?

I pair-programmed with an AI coding assistant. The AI acted as a rapid prototyping partner. We collaboratively debugged the opaque API errors by writing analysis scripts, identified the hidden AVIF file formats, and co-wrote the Pillow image resizing logic. The AI was also instrumental in designing the multi-key round-robin rotation system to overcome the API rate limits smoothly.

Q6: What was your approach to the multi-modal evidence review problem?

HackerRank Orchestrate - AI Judge Interview Prep

Our approach was to treat it as a pipeline: (1) Ingestion and preprocessing of images to guarantee compatibility, (2) Formatting a structured prompt with the claims data and the images encoded in base64, (3) Request routing through our quota-managed client to the Vision Language Model, and (4) Parsing the deterministic JSON output into the final predictions CSV. We aimed for reliability and throughput over complex multi-agent reasoning.

AI Judge Interview Preparation

Candidate: Rishabh Sharma | Project: Multi-Modal Evidence Review System

This document outlines strategic responses for your upcoming AI Judge interview. The answers are tailored to highlight the engineering rigor, resilience features, and system design decisions evident in your Python pipeline.

Q1: Can you walk me through your overall approach to building this Multi-Modal Evidence Review System?

My approach was heavily influenced by the need to build a deterministic and highly resilient Python pipeline around a non-deterministic AI model. The system operates in clear, sequential stages:

- **Data Ingestion & Preprocessing:** The system first parses the claims and dynamically resizes image bytes via Pillow to ensure we stay under the strict 30-million-pixel limit.
- **Contextual Prompting:** I consolidate all images, the user's chat history, and specific evidence requirements into a single, comprehensive prompt. This allows the Vision Language Model (VLM) to perform a holistic analysis rather than evaluating images in isolation.
- **Structured Output:** I force the model to output strict JSON using Groq's JSON mode, integrating a Chain-of-Thought "reasoning" step so the model evaluates severity and visual evidence before arriving at a final status.
- **Deterministic Post-Processing:** Finally, I pass the raw VLM output through a strict validation function (``enforce_consistency``) that overwrites any model hallucinations by applying rigid business rules.

Q2: How did you handle reliability, rate limits, and cost constraints in your architecture?

Because I optimized the architecture to run entirely on the free tier (\$0 cost), handling rate limits and API failures was a primary technical challenge. I built a custom wrapper, the `GroqClient`, to manage this automatically:

- **API Key Rotation:** The client accepts a comma-separated list of keys and automatically rotates to the next one if quotas are exhausted.
- **Model Fallback Chain:** If the primary Llama 4 Scout model fails or hits limits, the system seamlessly falls back to the Llama 4 Maverick model without crashing.
- **Exponential Backoff:** For transient 503 errors, I implemented a retry loop with exponential backoff, up to 5 retries, before exhausting a model.
- **Rate Limiting:** I enforced a hard 2.0-second delay between requests to ensure the system stays safely within the 60 Requests-Per-Minute free-tier limit.

Q3: Large Language Models can sometimes hallucinate or be inconsistent. How did you ensure the system's outputs align with the business requirements?

I tackled model inconsistency from two directions: prompt engineering and deterministic code constraints.

On the prompting side, I provided explicit definitions for ambiguous categories—for instance, distinctly separating a "scratch" from a "dent", or "crack" from "glass_shatter". I also included Few-Shot examples to anchor the model's expected behavior on edge cases.

However, prompting isn't foolproof. To guarantee business logic adherence, I wrote the `enforce_consistency` Python function. This acts as an absolute source of truth. For example, if the model flags `valid_image` as false, my code automatically forces `evidence_standard_met` to false. If `evidence_standard_met` is false, it forces the claim status to `not_enough_information`. This hardcoded layer prevents any contradictory outputs from reaching the final CSV.

Q4: How did you utilize AI tools while developing and coding this solution?

In my typical VS Code development workflow for APIs and data pipelines, I prefer to architect the core control flow and logic myself, but I leverage AI assistants as highly capable pair programmers for specific utility tasks.

For this system, I used AI to help brainstorm the edge cases injected into the prompt's Few-Shot examples, ensuring scenarios like "cropped or obstructed images" or "claim mismatches" were well-represented. I also used AI to help optimize the Pillow-based image resizing function, specifically to quickly determine the correct Lanczos scaling math required to compress images optimally without exceeding the API's payload and pixel limits.

Q5: What challenges did you face with the multi-modal aspect (images + text), and how did you resolve them?

One major challenge was dealing with corrupted or exceptionally large images that would trigger 400 or 413 HTTP errors, breaking the pipeline. I realized that retrying a structurally invalid image is a waste of API calls.

To resolve this, I implemented a pre-flight image processor. The script inspects the byte size and pixel count before the request is even made. If it's too large, it dynamically compresses the JPEG quality or downscales the dimensions. Furthermore, in the exception handling block, I specifically catch "invalid image data" or "413" errors to abort retries instantly for that specific claim, logging a fallback row instead of getting stuck in an infinite retry loop.